Student Reflection Notes and Project Retrospective

Cloud Computing Mini Project (CS3081)

Name: Rachit Srivastava

Roll number: 230101085

Project title: Knowledge Distillation and Deployment of a Lightweight Command-Parsing Model

1. Project Overview

The main aim of this project was to develop a small AI model that can understand English commands and convert them into JSON format.

For example, when a user says "set an alarm for 7am tomorrow", the model identifies the user's intention, date and time.

We used a larger AI model called Phi-3.5 as a teacher and a smaller model called Pythia-70M as a student. The student learns from the teacher's responses. This process is called knowledge Distillation.

The trained model is then deployed using Docker and Kubernetes on a local cloud environment.

2. What I Learned

Through this project, I learned several new concepts!

i) Knowledge Distillation: How a smaller AI model can learn from a larger model and perform a specific task using fewer resources.

ii) Model Training: How to prepare datasets, train an AI model, and evaluate its accuracy using Python and Pytorch.

iii) Docker: How to package an application with its dependencies so it can run in different environments.

iv) Kubernetes: How to manage containers, distribute requests, and restart applications when they fail.

(v) Cloud Storage: How SeaweedFS can be used to store datasets and trained AI models.

vi) Networking: How subnetting and network access rules help manage and secure ~~not~~ network traffic.

This project helped me understand how AI and cloud computing can work together.

3. Challenges Faced

While working on this project, I came across several technical challenges.

→ Challenge 1: Limited Hardware Resources

Training AI models requires considerable memory and processing power. Running these models on a normal laptop without a dedicated GPU was challenging.

Solution: I used a smaller student model, CPU based training, and suitable training settings to reduce resource usage.

→ Challenge 2: Incorrect AI responses

Sometimes, the teacher model can generate incorrect JSON or values that do not match the input command.

Solution: I used data-cleaning and validation scripts to remove invalid responses, incorrect fields, and duplicate commands before training.

→ Challenge 3: Docker Image Size

Machine learning libraries can make Docker images very large, increasing storage requirements.

Solution: I used CPU-only dependencies and kept the trained model outside the Docker image in object storage.

→ challenge 4: Application startup and failures

The AI services need time to download and load the model. Requests arriving before the model is ready can fail.

Solution: I used kubernetes health and readiness checks so the service accepts requests only when the model is ready.

→ challenge 5: Network security

Understanding subnet allocation and firewall rules was initially difficult.

Solution: I studied subnetting and used the provided NACL simulator to check whether network packets were allowed or blocked correctly.

These challenges helped improve my practical problem solving and debugging skills.

4. Key Findings and Observations

During this project, I understood some important points:

i) A smaller AI model can learn a specific task from a larger model through knowledge distillation.

ii) Pythia-70M has about 54 times fewer parameters than the Phi-3.5 teacher model, making it more suitable for lightweight deployment.

iii) Correct data cleaning is important because incorrect training examples can affect model performance.

iv) Docker and kubernetes make application deployment and management easier.

v) kubernetes health checks, scaling and recovery features help improve application reliability.

vi) Performance testing is ~~importa~~ important to understand response time, memory usage, and the limitations of a deployed application.

5. Possible Future Improvements

There are several ways this project can be improved!

- Improve model Accuracy: Use better quality teacher responses and improve the training process.
- Reduce Memory Usage: Apply model quantization to make the student model even lighter.
- Improve Performances: Optimize inference to reduce response time.
- Better Monitoring: Add monitoring tools to track CPU, memory, errors, and application health.
- Better scalability: Test the application on multiple machines instead of only one local laptop.

6. Conclusion

Overall, this project helped me gain practical knowledge of machine Learning, Docker, Kubernetes, cloud storage, and networking.

The most important thing I learned is how a small AI model can be trained using a larger model and deployed as a service with limited computing resources.

It also helped me understand the complete process of building, testing, deploying and managing an AI application.

Github:

https://github.com/Rachit4Srivas/cloud-command-parser

Signature: Rachit Srivastava

Date: 6 November 2026